

Layered Bitemporal Graphs: Statement Identity, Two Clocks, and Metagraphs as Tags

Volodymyr Pavlyshyn

Independent researcher

Revision 4, 3 October 2026

github.com/Volland

Abstract

Agent memory needs durable statement identity, provenance, and a distinction between when a claim held and when the system recorded it. We study layered bitemporal graphs, in which statement occurrences can reference other occurrences and carry both transaction-time lifetimes and valid-time intervals. On a referentially complete history, reference-closed sets form an Alexandrov topology. Its interior operator, the ground core, characterises minimal retraction cascades, grounded snapshots, and effective valid-time intervals. We give a correction contract that preserves structural closure under explicit patch and lineage conditions; preservation of references excludes changes made to the corrected root. A typed anchor-and-role encoding of metagraphs commutes with well-formed bitemporal snapshots. Compact binary encodings require additional acyclicity for insertion under a live-reference discipline. The calculus is realised in Tiramemsu, an embedded SQLite graph database. Retained checks identified four implementation gaps; endpoint validation, closed retention and transaction-date guards now prevent those counterexamples. These checks support selected contracts rather than certifying the entire implementation. Our contribution is an operational account of recursive temporal reference integrity, rather than a new theory of temporal databases or finite topologies.

1 Introduction

Consider an agent that reads in a chat log that Alice works at Acme. It records the fact, notes that it is 80% confident, that the source was a particular conversation, and that a belief it holds elsewhere is supported by this fact. A month later, Alice says she actually joined in February, not January. A week after that, an auditor asks what the agent believed on the 3rd of March, on what evidence, and since when.

Each part of this scenario has a mature answer in some literature. Annotating a fact needs statement-level identity, as in reification, RDF-star and RDF 1.2 [22, 23], singleton properties [37], Wikidata qualifiers [24], the Amazon Neptune OneGraph model [34] and the domain graph of MillenniumDB [47]. Distinguishing *when it was true* from *when we knew* is the bitemporal model of temporal databases [29, 30, 43, 44], standardised in SQL:2011 [33]. Historical versions are queryable under normal operations in Datomic [11] and XTDB [50]; explicit history-removal operations, such as XTDB’s ERASE, fall outside our never-forget discipline. Withdrawing everything that rests on a withdrawn premise is the job of truth maintenance systems [12, 14]. Grouping facts into larger units that can themselves be related is the metagraph [4, 9, 16, 18].

What is less studied is what happens when all of these live in *one* structure, at the granularity of a single statement. Then annotations have lifetimes, corrections must carry annotations along, a valid-time slice can contain a confidence score whose fact is not valid at that instant, and a container can be retracted while its contents stay. This paper gives that combined structure a precise semantics and proves the properties that make it safe to build on.

Contributions.

1. A formal model of *layered bitemporal graphs* (LBGs): statements with identity, a reference relation between statements, transaction-time lifetimes and valid-time intervals (Sections 3 and 4).
2. An application of the standard down-set construction [3]: the *ground core* γ identifies grounded sets with the open sets of an Alexandrov topology (Lemma 1).
3. Theorems on the operations: stability of time travel (Theorem 3), minimality of the retraction cascade (Theorem 4), groundedness of snapshots as lifetime nesting (Theorem 5), effective validity (Theorem 7), and correctness of correction by cascade-and-replay (Theorem 8) (Section 5).
4. A faithful encoding of temporal metagraphs into LBGs and the *Bitemporal Lifting Theorem* (Theorem 10), with corollaries for unfold, nesting, fold and container retraction (Section 6).
5. A realisation in an embedded database and a conformance analysis: four conformance repairs, with retained regression checks and explicit evidence limits (Section 8).

Terminology. *Layered* here means stratified by reference: a statement about a statement sits one layer above it. This is not the *multilayer network* of network science [32], where layers are parallel edge sets over a shared vertex set. It is close to the *multilayer graphs* of Angles et al. [2], in which edges have identifiers and can be endpoints of other edges; we add the two clocks and the operations that keep such structures consistent over time.

2 Background and Related Work

Bitemporal data. Snodgrass and Ahn separated *valid time*, when a fact holds in the modelled world, from *transaction time*, when it is current in the database [44]. The Bitemporal Conceptual Data Model attaches to each tuple a set of bitemporal chronons and shows that several representational models are snapshot-equivalent views of it [29]. The consensus glossary fixes the vocabulary [30], and SQL:2011 introduced application-time and system-versioned tables [33]. Our lifetimes and valid intervals follow these two time dimensions. Temporal referential integrity already occurs in flat relations: a child period must be covered by matching parent periods [33, Section 2.2.2]. Here references are to individual immutable occurrences, and may nest recursively. We specialise that established integrity problem to occurrence references, structural cascades, and correction by replay.

Temporal graphs. Network science studies graphs whose edges are active at times, and time-respecting paths along them [8, 26, 31]. Wu et al. give efficient algorithms for earliest-arrival, latest-departure, fastest and shortest temporal paths [49]. In databases, temporal property-graph models attach intervals to nodes and edges [13, 42] and temporal RDF attaches them to triples [21]. These cited models motivate temporal graph queries. Our comparison concerns their published representations, rather than an impossibility claim about encodings: ordinary reification can represent higher-order dependencies. We make those dependencies explicit at the level of temporal statement occurrences and study their closure under correction. Systems work on temporal graph stores, such as AeonG [27], Clock-G [35] and ChronoGraph [6], optimises the storage and traversal of versioned vertices and edges, and temporal graph algebra [36] gives point- and interval-based semantics for evolving graphs.

Statement-level identity. RDF reification, RDF-star [22] and the RDF 1.2 reifier model [23] let a triple be talked about. A quoted triple identifies content, not by itself an occurrence; RDF 1.2 reifiers are resources that relate to triple terms. The cited RDF 1.2 document is a Candidate Recommendation Snapshot, not a final Recommendation. Singleton properties mint a predicate per occurrence [37]. Hernández, Hogan and Krötzsch compared these encodings for Wikidata’s qualified statements [24, 46], and benchmarks show the cost of encoding statement metadata on top of plain triples [15, 38]. Named graphs group triples under a name that can carry provenance [7]. OneGraph [34] and MillenniumDB’s domain graph [47] make every edge an object with an identifier. Gelling, Fletcher and Schmidt’s *statement graphs* [17] make statement identity the unifying primitive of RDF, RDF-star and property graphs, and are the closest data model to ours; Angles et al.’s multilayer graphs [2] are another. Their cited presentations emphasise graph-model interoperability; we study an additional temporal correction contract rather than claiming that those representations cannot encode time or containment. In our model, each *occurrence* of a statement has an identifier, properties included. Unlike RDF 1.2, where a reifier may stand for several triples and a triple for several reifiers, our identifier is a total function to content.

Metagraphs and hypergraphs. Basu and Blanning define a metagraph over a generating set with edges between *sets* of elements [4]. Chernenkiy, Gapanyuk and colleagues add metaverices and metaedges that contain fragments, and annotations that are themselves metagraphs [9, 16]. Goertzel develops morphism-based folding and unfolding [18]; our membership-tag operation below is a narrower storage operation, not a claim to implement that entire algebra. HypergraphDB stores hyperedges that can point to hyperedges [28]. The use of metagraphs for agent memory is discussed in the author’s self-published book [40]. Definition 9 combines set-valued endpoints, higher-order atoms and membership occurrences; it is not an identification of these traditions. We establish snapshot lifting for this explicit variant under temporal well-formedness (Theorem 10).

Truth maintenance and belief revision. A justification-based TMS withdraws every belief whose justification loses support [14]; an assumption-based TMS tracks the assumption sets under which beliefs hold [12]. AGM belief revision axiomatises contraction and revision of theories [1]. Our cascade is a syntactic, monotone cousin of TMS withdrawal. Support is structural (a layer rests on its base), there are no alternative justifications, and nothing is forgotten, because withdrawal closes a lifetime. Theorem 4 plays the role of a minimal-change postulate: the cascade withdraws exactly what it must and nothing more.

Provenance. Why- and where-provenance [5] and provenance semirings [19] annotate query answers with the source tuples that produced them. Statement identity makes such citations durable, because a cited statement can later be checked for retraction in any snapshot.

Agent memory. Recent agent-memory systems store temporal knowledge graphs whose edges carry validity windows and system times; Zep’s Graphiti is a prominent example [41]. Other memory layers manage what enters a bounded context [39], extract and consolidate memories [10, 51], or retrieve over a knowledge graph [20]; benchmarks such as LongMemEval test knowledge updates and temporal reasoning explicitly [48]. Our analysis complements these cited agent-memory architectures with an explicit contract for structural annotation replay; we do not claim an exhaustive comparison against every system version. The surveys of knowledge graphs [25] summarise the broader design space. Our model can be read as giving such systems a precise account of what a correction, a retraction or a snapshot does to the annotations on a fact.

Cited representation	Established mechanism	Relation to this paper
SQL:2011 [33]	Temporal periods and referential constraints	Predecessor for temporal coverage; our targets are immutable occurrences with recursive dependencies.
Statement graphs [17]	Identity and cross-model mappings	Closest structural model; we add a scoped temporal operation contract.
Multilayer graphs [2]	Identified edges as endpoints	Shared higher-order primitive; we analyse reference closure across two clocks.
Graphiti/Zep preprint [41]	Temporal agent-memory architecture	Application context; no completeness or impossibility claim about later versions.

Table 1: Comparison against the cited publications, not all possible encodings or later releases.

Symbol	Meaning
S, κ	Complete occurrence history and immutable triple content
X, V_t, V^v	A row set, as-of view, and valid-at slice
$\rightarrow, \rightarrow_\Delta$	Full references and references excluding lineage predicates
γ, γ_Δ	Full and structural ground cores
$\uparrow_X^r$	Full reference dependents inside the induced live graph
λ, ν	Transaction lifetime and claimed valid interval
C, F, R, σ	Cascade, dropped memberships, retained set, fresh-copy map

Table 2: Notation. Lineage still requires allocated historical targets.

3 Layered Graphs

3.1 Statements and references

Fix disjoint countable sets: nodes $\mathcal{N}$, literals $\mathcal{L}$, statement identifiers $\mathcal{E}$ and transaction numbers $\mathbb{T} = \{1, 2, \dots\}$. Predicates are nodes, $\mathcal{P} \subseteq \mathcal{N}$.

Definition 1 (Layered graph). A *layered graph* is a finite set $S \subseteq \mathcal{E}$ of statements together with a content function $\kappa: S \rightarrow (\mathcal{N} \cup \mathcal{E} \cup \mathbb{T}) \times \mathcal{P} \times (\mathcal{N} \cup \mathcal{L} \cup \mathcal{E} \cup \mathbb{T})$, written $\kappa(e) = (s_e, p_e, o_e)$, such that no statement mentions itself: $s_e \neq e \neq o_e$, and every statement-valued subject or object belongs to S . Thus the history is *referentially complete*; its views need not be.

The identifier e is the identity of one *occurrence* of a fact. Two statements may have equal content (parallel edges); content is immutable. A statement whose object is a literal is a property; one whose subject or object is a statement is a layer or a citation.

Definition 2 (References, support, dependents). For $e, e' \in S$ write $e \rightarrow e'$ if $e' \in \{s_e, o_e\}$. Let $\rightarrow^*$ be the reflexive-transitive closure. The *support* of e is $\downarrow e = \{e' \in S : e \rightarrow^* e'\}$, everything e rests on, e included. For $X \subseteq S$ the *dependents* of e in X are $\uparrow_X e = \{x \in X : x \rightarrow_X^* e\}$, where $\rightarrow_X$ is the reference relation induced on X . On a grounded X , this equals reachability in the whole history restricted to starting points in X .

The relation $\rightarrow$ may contain cycles of length at least two (statement a about b about a). Proposition 6 shows that cycles cannot arise under the live-reference discipline of Section 5. Where $\rightarrow$ is acyclic the *rank* of a statement, 0 if it references no statement and otherwise one more than the largest rank it references, is the layer it sits in.

3.2 Grounded sets and the ground core

The central notion instantiates the established correspondence between finite preorders and Alexandrov spaces [3]. We claim the operational application below, not the topology construction

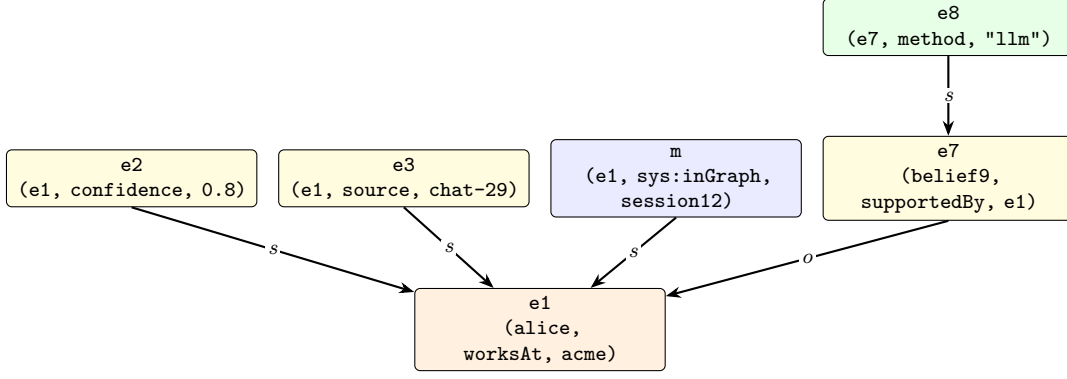


Figure 1: A layered graph. Rows are ranks 0, 1 and 2. Arrows are the reference relation $\rightarrow$, labelled by the position (subject or object) that holds the referenced statement. The support of **e8** is $\{\mathbf{e8}, \mathbf{e7}, \mathbf{e1}\}$; the dependents of **e1** are all six statements. The membership **m** places **e1** in a named graph, and is itself a statement.

itself.

Definition 3 (Grounded set, ground core). A set $X \subseteq S$ is *grounded* if $\downarrow e \subseteq X$ for every $e \in X$, that is, if $e \in X$ and $e \rightarrow e'$ imply $e' \in X$. The *ground core* of an arbitrary $X \subseteq S$ is

$$\gamma(X) = \{e \in X : \downarrow e \subseteq X\}.$$

A grounded set never contains an annotation without the statement it annotates, nor a citation without what it cites. A raw engine state with an unallocated endpoint is outside Definition 1. For such a state, let B be the rows with a missing statement-valued endpoint. Remove B and all its reverse-reachable dependents before applying γ to the remaining, referentially complete carrier. This separate integrity check prevents missing identifiers from disappearing from the formal reference relation.

Lemma 1 (Ground core). *For all $X, Y \subseteq S$ and families $(X_i)_{i \in I}$:*

1. $\gamma(X) \subseteq X$, $\gamma(\gamma(X)) = \gamma(X)$, and $X \subseteq Y$ implies $\gamma(X) \subseteq \gamma(Y)$;
2. $\gamma(X)$ is grounded, and it is the largest grounded subset of X ;
3. $\gamma(\bigcap_i X_i) = \bigcap_i \gamma(X_i)$;
4. arbitrary unions and intersections of grounded sets are grounded.

Hence the grounded sets are the open sets of an Alexandrov topology on S , and γ is its interior operator.

Proof. (1) Deflation and monotonicity are immediate. If $e \in \gamma(X)$ and $e' \in \downarrow e$ then $\downarrow e' \subseteq \downarrow e \subseteq X$ by transitivity, so $\downarrow e \subseteq \gamma(X)$; this shows $e \in \gamma(\gamma(X))$ and also that $\gamma(X)$ is grounded. (2) If $Y \subseteq X$ is grounded and $e \in Y$, then $\downarrow e \subseteq Y \subseteq X$, so $e \in \gamma(X)$. (3) $\downarrow e \subseteq \bigcap_i X_i$ iff $\downarrow e \subseteq X_i$ for all i . (4) The defining condition is preserved by arbitrary unions and intersections. A topology closed under arbitrary intersections is Alexandrov; its open sets here are the down-sets of the preorder $\rightarrow^*$. $\square$

Lemma 2 (Cascade as a core). *If X is grounded and $e \in X$, then $X \setminus \uparrow_X e = \gamma(X \setminus \{e\})$.*

Proof. Let $x \in X \setminus \{e\}$. Because X is grounded, $\downarrow x \subseteq X$, so $\downarrow x \subseteq X \setminus \{e\}$ iff $e \notin \downarrow x$ iff $x \notin \uparrow_X e$. The statement e itself lies in $\uparrow_X e$ and not in $X \setminus \{e\}$. $\square$

4 Two Clocks

4.1 Lifetimes and intervals

Transaction numbers are gap-free and strictly increasing, one per committed transaction. Valid-time instants $\mathbb{V}$ form a total order (epoch milliseconds in the implementation); $\pm\infty$ denote unbounded ends. All intervals are half-open.

Definition 4 (Layered bitemporal graph). A *layered bitemporal graph* (LBG) is a layered graph (S, κ) with two further functions on S : a *lifetime* $\lambda(e) = [t_{\text{add}}(e), t_{\text{ret}}(e))$ with $t_{\text{add}}(e) \in \mathbb{T}$ and $t_{\text{ret}}(e) \in \mathbb{T} \cup \{\infty\}$, $t_{\text{add}}(e) \leq t_{\text{ret}}(e)$; and a *valid interval* $\nu(e) = [v_{\text{from}}(e), v_{\text{to}}(e)) \subseteq \mathbb{V}$, non-empty. Content $(\kappa(e), \nu(e))$ is immutable; only $t_{\text{ret}}(e)$ may change, once, from ∞ to a transaction number.

A statement with $t_{\text{add}}(e) = t_{\text{ret}}(e)$ was asserted and retracted inside one transaction and is visible in no snapshot. Changing a valid interval is a correction (Section 5.5), never an update.

Definition 5 (Views). For $t \in \mathbb{T}$ and $v \in \mathbb{V}$:

$$V_t = \{e \in S : t \in \lambda(e)\}, \quad V^v = \{e \in S : v \in \nu(e)\}, \quad V_t^v = V_t \cap V^v.$$

V_t is the *as-of* view, V^v the *valid-at* slice, and V_t^v the bitemporal snapshot. The *history* view is S itself.

A query evaluated on a view sees the content of exactly the statements in the view. The implementation lets each pattern of a query choose its own view; here one view per query is enough.

4.2 Histories and stability

A database evolves by committed transactions. Let $S^{(t)}$ denote the LBG after transaction t , with $t_{\text{ret}} = \infty$ for statements still live. A transaction t may only (a) insert statements with $t_{\text{add}} = t$ and fresh identifiers, and (b) set $t_{\text{ret}}(e) = t$ for statements live at that operation, including ones inserted earlier within t . Content remains immutable. This is the *never-forget* discipline: no row is deleted and no identifier is reused. The engine supplies the transaction protocol. Database triggers prevent deletion, replacement and content changes. Format 3 also validates current-transaction dates against the committed watermark, assuming the schema and engine metadata remain intact (Section 8).

Theorem 3 (Stability of time travel). *For all $t \leq t' \leq t''$, the view V_t computed in $S^{(t')}$ equals V_t computed in $S^{(t'')}$, statement for statement and with equal content. Moreover, V_t equals the set obtained by replaying, in order, the insert and retract events with transaction number at most t , processing each occurrence's addition before its retraction when their transaction numbers agree.*

Proof. Every transaction $u \in (t', t'']$ inserts statements with $t_{\text{add}} = u > t$, which are not in V_t , and sets $t_{\text{ret}}(e) = u > t$ on statements that were live, which keeps $t \in \lambda(e)$ exactly when it held before. Content never changes. For the second claim, a statement is in the replayed set iff its insert event has time $\leq t$ and its retract event, if any, has time $> t$, which is the definition of V_t . $\square$

The theorem is what makes an answer *about the past* reproducible: a snapshot never changes once its transaction has committed. The second half is checked in the implementation by property-based tests over random operation sequences.

5 Operations and Their Guarantees

5.1 Assert and create

$\text{Assert}(s, p, o, \nu)$ returns an existing live occurrence with the same triple and an overlapping valid interval, or inserts a fresh one. The implementation chooses the smallest matching eid. Returning an existing occurrence does not extend its interval to cover the requested interval; callers that need a distinct episode use *create* or an explicit correction. Here “same triple” excludes the valid interval, which is also immutable content. *Create* always inserts, which is how parallel edges are made. Both insert statements whose subject or object may be another statement. What they are allowed to reference decides whether snapshots stay grounded.

Definition 6 (Live-reference discipline). Let $\Lambda \subseteq \mathcal{P}$ be a set of *lineage* predicates. A transaction satisfies the *live-reference discipline* (LR) if every statement e it inserts with $p_e \notin \Lambda$ references only statements that are live at the moment of insertion (inserted earlier and not retracted).

Write $\rightarrow_\Lambda$ for the restriction of $\rightarrow$ to statements whose predicate is not in Λ , and call a set *structurally grounded* when it is grounded for $\rightarrow_\Lambda$. Write $\downarrow_\Lambda$, $\uparrow_{\Lambda, X}$ and γ_Λ for support, induced dependents and the core computed with this relation. All results about γ apply to this relation as well. Full reference closure and structural closure are different: lineage links may point to historical occurrences outside the live view, but never to missing identifiers. LR imposes liveness only on non-lineage references; all endpoints must already exist except during an explicitly specified atomic replay.

5.2 Retraction and its cascade

Retracting e retracts, in the same transaction, e and every live statement that transitively references it through a subject or object position: its layers, the statements that cite it, the layers on those, and so on. Nodes never cascade.

Theorem 4 (Cascade minimality). *Let X be the live set before a retraction of $e \in X$, and let X be grounded. The cascade retracts exactly $\uparrow_X e$, and:*

1. $X \setminus \uparrow_X e = \gamma(X \setminus \{e\})$, so the remaining live set is grounded;
2. $\uparrow_X e$ is the least set R such that $e \in R \subseteq X$ and $X \setminus R$ is grounded.

Proof. The cascade is a breadth-first search from e over inverse references among live statements, with a visited set, so it terminates on cycles and reaches exactly $\uparrow_X e$. Claim 1 is Lemma 2. For Claim 2, let R be any such set. Then $X \setminus R$ is a grounded subset of $X \setminus \{e\}$, hence contained in its core $X \setminus \uparrow_X e$ by Lemma 1(2), so $\uparrow_X e \subseteq R$. $\square$

For structural closure the theorem applies to $\rightarrow_\Lambda$: its minimal retraction is $\uparrow_{\Lambda, X} e$. The engine follows all live references, including lineage, and may retract more than this structural minimum. The full-reference minimality claim therefore requires full groundedness.

Under its stated hypothesis, the theorem says that the cascade is forced. Any retraction that leaves memory grounded must take at least $\uparrow_X e$ with it, and the cascade takes nothing else. In belief-revision terms it is the minimal contraction for the structural support relation. The same set can be computed without the writer, as a read on any view: as-of t it shows what depended on e then. It is also the set of ends of the regular path $(\hat{\text{sys}}:\text{subject} \mid \hat{\text{sys}}:\text{object})^*$ from e . Both equivalences are checked by property tests in the implementation.

5.3 Snapshots are grounded exactly when lifetimes nest

Theorem 5 (Transaction-time nesting). *For a history S the following are equivalent:*

1. *every as-of view V_t is structurally grounded;*
2. *for every structural reference $e \rightarrow_\Lambda e'$, $\lambda(e) \subseteq \lambda(e')$.*

Starting from the empty history, if insertions satisfy LR, retractions follow all live references, and corrections satisfy Theorem 8, then both hold.

Proof. (1) $\Leftrightarrow$ (2): V_t is grounded for every t iff for each reference $e \rightarrow_\Lambda e'$ and each t , $t \in \lambda(e)$ implies $t \in \lambda(e')$, which is $\lambda(e) \subseteq \lambda(e')$. For the sufficient condition we show by induction over the operations of each transaction that the live set stays structurally grounded; the committed live set after t is V_t , because a statement inserted and retracted within t has an empty lifetime. Insertion under LR adds a statement whose structural referents are live. Retraction preserves groundedness by Theorem 4, applied to $\rightarrow_\Lambda$ -dependents (the implementation's cascade follows all references, lineage included; the remaining set is still structurally grounded, because if $x \rightarrow_\Lambda y$ and y is removed then $x \rightarrow y \rightarrow^* e$, so x is removed too). Correction is covered by Theorem 8. $\square$

Condition 2 gives a geometric reading of a grounded history. Draw each lifetime as a segment on the transaction axis; a layer's segment lies inside the segment of the statement it rests on. Lineage links are the opposite case: their segment begins exactly where the referenced statement's segment ends.

Example 1 (Why LR is needed). Assert $e_1 = (\text{alice}, \text{worksAt}, \text{acme})$ at $t = 1$, retract it at $t = 2$, and at $t = 3$ assert $e_2 = (e_1, \text{confidence}, 0.8)$. Then $e_2 \in V_3$ but $e_1 \notin V_3$: the as-of view at 3 holds a confidence about nothing. Condition 2 fails, since $\lambda(e_2) = [3, \infty) \not\subseteq [1, 2) = \lambda(e_1)$.

Proposition 6 (Acyclicity). *Starting from the empty history, suppose ordinary and lineage insertions reference only already-existing identifiers. Suppose correction copies preserve an acyclic reference graph and the patched root adds no reference inside its replay set. Then the full history is acyclic and rank is well defined. A patch whose new statement-valued object belongs to $X \setminus C$ meets the latter condition.*

Proof. Ordinary insertion cannot close a cycle because existing immutable statements cannot mention its fresh identifier. Replay copies an acyclic induced graph; references leaving the copies point to old identifiers, none of which can point back to a fresh copy. Removing or replacing a root reference by an old target outside C does not add an internal cycle. The new lineage statement is inserted last and refers only to existing identifiers. Induction proves the claim. $\square$

LR ensures existing targets for ordinary insertions, but lineage insertion must also reject unallocated targets. An arbitrary patch into its own replay set can create a cycle and is not covered by this proposition.

Under acyclicity, the cascade of e contains only statements of strictly higher rank than e (apart from e): retraction propagates upward through the layers and never sideways.

5.4 Valid time does not nest, and how to read it

Valid intervals are content chosen by the writer, and a layer's interval is independent of its base's. This is deliberate: a confidence assessed in 2026 about a fact that held in 2020 is valid in 2026. Consequently V^v , and with it V_t^v , need not be grounded even when every V_t is. A reader who wants a grounded slice can take its core, and the core has a closed form.

Definition 7 (Effective validity). For a view X and $e \in X$ let $\nu_X^*(e) = \bigcap \{\nu(e') : e' \in \downarrow e\}$ if $\downarrow e \subseteq X$, and $\emptyset$ otherwise.

An intersection of half-open intervals is a half-open interval, possibly empty, so effective validity is again an interval and can be stored or indexed like any other.

Theorem 7 (Effective validity). *For every view X and instant v :*

1. $\gamma(X \cap V^v) = \{e \in X : v \in \nu_X^*(e)\}$, the largest grounded subset of the slice;
2. ν_X^* is the greatest fixpoint, under pointwise inclusion, of $F(\varphi)(e) = \nu(e) \cap \bigcap_{e \rightarrow e'} \varphi(e')$ on the complete lattice $\prod_{e \in \gamma(X)} 2^{\mathbb{V}}$ ordered pointwise by inclusion.

Proof. (1) $e \in \gamma(X \cap V^v)$ iff every $e' \in \downarrow e$ lies in X and has $v \in \nu(e')$, iff $v \in \nu_X^*(e)$. Maximality is Lemma 1(2). (2) Because $\downarrow e = \{e\} \cup \bigcup_{e \rightarrow e'} \downarrow e'$, ν_X^* is a fixpoint. If φ is any fixpoint then $\varphi(e) \subseteq \nu(e)$ and $\varphi(e) \subseteq \varphi(e')$ whenever $e \rightarrow e'$, so following any reference path from e to e' and chaining these inclusions, $\varphi(e) \subseteq \nu(e')$ for every $e' \in \downarrow e$ (cycles are harmless, since only finite paths are used), that is $\varphi \subseteq \nu_X^*$. The fixpoint also exists by Tarski's theorem [45], since F is monotone on a complete lattice; the closed form makes the theorem constructive and valid in the presence of cycles. $\square$

Theorems 5 and 7 show an asymmetry between the clocks. Transaction time nests under the stated LR and operation contracts. Valid time is the writer's claim about the world and nests only if the reader asks for it, through γ . The asymmetry is a design choice, and the closed form reduces the reader's operation to: ν^* is an interval intersection along the support, computable during the same traversal that evaluates a layered pattern.

5.5 Correction by cascade-and-replay

Correction retracts a live root r and its full live-reference cascade $C = \uparrow_X r$, then replays retained occurrences under fresh identifiers. A foreign membership is a membership in C whose container is outside C . Let F be the foreign memberships selected for dropping. Retention must be closed under structural references inside C :

$$R = \gamma_{\Lambda, C}(C \setminus F), \quad \gamma_{\Lambda, C}(Y) = \{e \in Y : \downarrow_{\Lambda, C} e \subseteq Y\},$$

where support is computed in the structural reference graph induced on C . Lineage references do not force retention and may continue to name historical occurrences. Choose a bijection $\sigma : R \rightarrow R^+$ onto fresh identifiers and extend it to a substitution f that fixes all identifiers outside R . In particular, dropped occurrences retain their old historical identifiers; we do not create references to fresh identifiers for copies that were never inserted.

Definition 8 (Admissible correction). A correction is admissible if X is structurally grounded in a referentially complete history, $r \in R$, and the patched root has no direct self-reference and has a non-empty valid interval. Every patched statement-valued endpoint must exist in the extended history. If the root predicate is not lineage, every patched statement-valued endpoint must belong to $(X \setminus C) \cup \sigma(R)$ after substitution. Subjects and predicates are preserved modulo f ; the root object and valid interval may change. The operation is atomic, adds only the replayed rows and one fresh lineage row, and performs no other schema-induced retractions during replay.

Theorem 8 (Correction). *For an admissible correction at transaction t , replay every retained subject and object through f , apply the admissible patch to the root, and insert $\ell = (\sigma(r), \text{sys}:\text{supersedes}, r)$ with $\text{sys}:\text{supersedes} \in \Lambda$. Then the new live set is $X' = (X \setminus C) \cup \sigma(R) \cup \{\ell\}$, and:*

1. For $x \in R \setminus \{r\}$ and $y \in R$, $x \rightarrow y$ iff $\sigma(x) \rightarrow \sigma(y)$. Non-root content and valid intervals are preserved modulo f . The root's references are preserved only where the patch leaves its endpoints unchanged modulo f .

2. X' is structurally grounded and the extended history is referentially complete.
3. Every previously committed as-of view is unchanged. In particular, $C \cap V_{t-1} \subseteq V_{t-1}$ retains its original content permanently; rows inserted earlier within t may belong to C without belonging to V_{t-1} .

For a validity-only patch all references within R are preserved, so the full induced reference structures on R and $\sigma(R)$ are isomorphic.

Proof. (1) For non-root rows both endpoint positions are substituted by the injective map f ; old external identifiers cannot equal fresh copy identifiers. This proves the stated equivalence. A root object patch need not preserve that relation. (2) If a surviving old row structurally referenced a removed row, it would lie in the full induced cascade C , a contradiction. For a non-root $x \in R$ and $x \rightarrow_{\Lambda} y$, structural groundedness gives $y \in X$. If $y \in C$, closure of R gives $y \in R$, so the copy references $\sigma(y) \in X'$. Otherwise $y \in X \setminus C$. The root meets this condition by admissibility; ℓ is lineage and imposes no structural dependency. Historical lineage targets remain allocated, and all new endpoints exist by construction or admissibility. (3) All new rows have addition time t and all retractions have time t . Theorem 3 preserves every committed view with time less than t . $\square$

Remark 1 (Dropped memberships and schema effects). If an implementation allocates a fresh id for every row of C but inserts only $C \setminus F$, a retained row referencing $m \in F$ can acquire a dangling target. This violates referential completeness even for lineage; structural closure also fails for a non-lineage reference. The specified substitution and retention rule avoid that behavior. A membership root that is itself dropped fails $r \in R$; its correction must be rejected or assigned an explicit alternative policy. Cardinality or uniqueness rules that remove other rows during replay need their own transaction contract; they are not covered by this theorem.

Replay preserves structural annotations, not their truth about the replacement. An unchanged confidence score or source citation may need revalidation after an object patch. Applications must select a retain, drop, or revalidate policy; this theorem describes retention of the chosen structurally closed set. Repeated correction gives a predecessor chain in history. Replayed lineage can also create shortcuts, and the complete chain need not exist in a live-only view.

Example 2 (A correction timeline). At $t = 1$ create $r = (\text{alice}, \text{worksAt}, \text{acme})$ valid on $[\text{Jan}, \text{Jun})$, a confidence layer a valid everywhere, a container anchor c , and membership $m = (r, \text{sys:inGraph}, c)$. At $t = 2$ correct only r 's interval to $[\text{Feb}, \text{Jun})$ and drop m . Then $C = \{r, a, m\}$, $F = \{m\}$ and $R = \{r, a\}$. The live view contains $\sigma(r), \sigma(a), c, \ell$, while V_1 still contains r, a, c, m . In the structural valid-at slice for January, $\sigma(r)$ is absent and γ_{Λ} removes $\sigma(a)$; for March both are retained. The old membership remains in history and is not copied to the replacement.

6 Metagraphs on Layered Graphs

A metagraph has two powers that a graph lacks [4, 9, 16, 40]. *Edges are vertices*: a relationship can be an endpoint of another relationship. *Vertices are containers*: a vertex, or an edge, can hold a subgraph. Statement identity gives the first power directly. This section shows that membership statements whose graph name may be a statement give the second, and establishes snapshot lifting under explicit temporal conditions.

6.1 Temporal metagraphs

Definition 9 (Metagraph). A *metagraph* is a tuple $M = (V, E, \text{src}, \text{tgt}, \text{lab}, \mu, \alpha)$ where V and E are disjoint finite sets of vertices and edges, $A = V \cup E$ is the set of *atoms*, $\text{src}, \text{tgt}: E \rightarrow 2^A \setminus \{\emptyset\}$

give each edge non-empty source and target sets with $x \notin \text{src}(x) \cup \text{tgt}(x)$, $\text{lab}: E \rightarrow \mathcal{P}$ labels edges, μ is a finite set of *memberships*, each with a member $\text{mem}(m) \in A$ and a container $\text{con}(m) \in A$, $\text{mem}(m) \neq \text{con}(m)$, and α is a finite set of *annotations*, each with a bearer $\text{bear}(a) \in A$, a key in $\mathcal{P}$ and a literal value.

The definition covers the classical variants. Basu–Blanning metagraphs are the case $\mu = \alpha = \emptyset$ with $\text{src}, \text{tgt} \subseteq V$; edges whose endpoints are edges give edges-as-vertices; a vertex with members is a metavertex; an edge with members is a metaedge in the sense of Gapanyuk; annotations give attributes. Hypergraphs take $\text{src} = \text{tgt}$. Atoms, memberships and annotations are collectively the *components* of M . Component identifiers are pairwise disjoint, so membership and annotation occurrences remain distinct even when their fields agree.

Each component c *depends on* the atoms it mentions: an edge on its endpoints, a membership on its member and container, an annotation on its bearer. A vertex depends on nothing.

Definition 10 (Temporal metagraph). A *temporal metagraph* is a metagraph M whose components each carry a lifetime $\lambda(c)$ and a valid interval $\nu(c)$. Its snapshot $H(t, v)$ keeps the components c with $t \in \lambda(c)$ and $v \in \nu(c)$. It is *well-formed* if, whenever c depends on d , $\lambda(c) \subseteq \lambda(d)$ and $\nu(c) \subseteq \nu(d)$.

Components are occurrences, like statements: re-adding a removed membership creates a new membership with its own lifetime. Well-formedness says that no component outlives what it mentions on either clock. It is precisely what makes every snapshot $H(t, v)$ a metagraph, since a snapshot of an ill-formed history can contain an edge whose endpoint is absent.

6.2 The encoding

Reserve the predicates `mg:Edge`, `mg:src`, `mg:tgt`, `sys:inGraph` and `rdf:type`, and the vertex marker node `mg:Vertex`. Annotation keys are outside this reserved predicate set. General edge labels may be any predicates. The encoding assigns each atom x an *anchor* statement $h(x)$, its handle, and encodes every other component as a layer on handles.

Definition 11 (Encoding). Given a temporal metagraph, `enc` produces the LBG with the following statements, each inheriting the lifetime and valid interval of the component it encodes:

- for a vertex x : $h(x) = (n_x, \text{rdf:type}, \text{mg:Vertex})$;
- for an edge x : the anchor $h(x) = (n_x, \text{mg:Edge}, \text{lab}(x))$ and the role statements
 $(h(x), \text{mg:src}, h(y))$ for $y \in \text{src}(x)$, $(h(x), \text{mg:tgt}, h(y))$ for $y \in \text{tgt}(x)$;
- for a membership m : $(h(\text{mem}(m)), \text{sys:inGraph}, h(\text{con}(m)))$;
- for an annotation a with key k and value d : $(h(\text{bear}(a)), k, d)$.

Here n_x is a fresh node per atom. Role statements inherit the edge’s lifetime and interval.

The membership statement is the shape introduced by statement graphs in the implementation: its object, the graph name, is the identifier of a statement, so a container can be a vertex or an edge alike. Membership is a *tag* on the member, not a copy, so an atom in several containers keeps one identity and one set of annotations.

Definition 12 (Encoding normal form). A static LBG is in encoding normal form if every row is exactly one of the four shapes in Definition 11. Vertex and edge anchors use their distinct predicates and a distinct node n_x per atom. Each role has an edge-anchor subject and an anchor object different from its subject; each edge anchor has at least one source and target role. There is exactly one role occurrence per (edge, role kind, endpoint). Memberships have distinct anchor

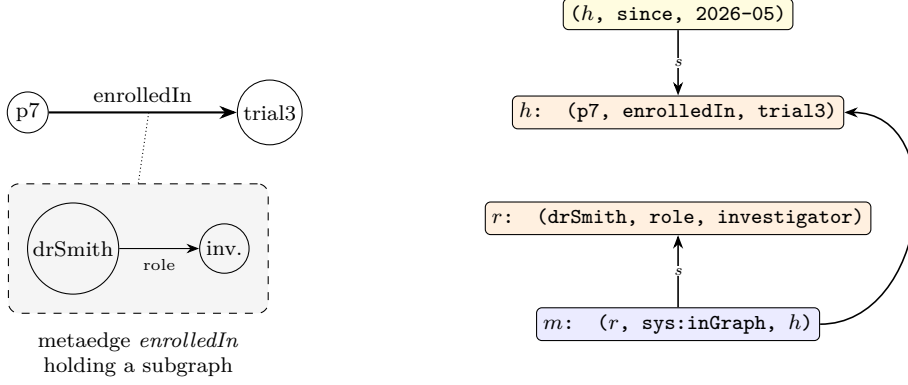


Figure 2: A metaedge holding a subgraph, and its encoding with the compact binary form of Remark 2. Retracting h cascades to the membership m and the annotation, and leaves the contained statement r live: the container goes, its contents stay (Corollary 12).

subjects and objects. Annotations have anchor subjects, non-reserved keys and literal objects. All referenced anchors exist. A temporal normal form additionally requires each role’s lifetime and valid interval to equal those of its edge anchor.

Definition 13 (Decoding). On encoding normal form, dec creates one vertex or edge per anchor. Role objects supply the edge’s source and target sets and its anchor object supplies the edge label. Every membership or annotation row creates its own component occurrence. In temporal normal form, clocks are read from the corresponding anchors or component rows. Role clocks need not be stored twice in the decoded model.

Theorem 9 (Faithful encoding). *For every metagraph M with non-reserved annotation keys, $\text{dec}(\text{enc}(M)) \cong M$. Conversely, for every static LBG G in encoding normal form, $\text{enc}(\text{dec}(G)) \cong G$. Thus the static image is exactly that normal form up to identifier renaming. The same roundtrips preserve all clocks for temporal normal form.*

Proof. The two anchor predicates distinguish atom kinds even when an edge label is mg:Vertex . Decoding assigns each anchor its atom and each role its unique endpoint in the appropriate set. Each membership and annotation retains its occurrence identity. This recovers M component by component. Conversely, normal-form role uniqueness prevents decoding from merging duplicate roles; node injectivity and the non-self constraints recover valid anchors and incidences. Every row is reconstructed by exactly one encoding clause. Role-clock equality makes the temporal reconstruction exact as well. $\square$

Predicate subject-kind declarations enforce some local shape constraints in the implementation. Role uniqueness, anchor existence, clock equality and non-empty source and target sets require additional validation; the current engine does not enforce the complete encoding normal form automatically.

Remark 2 (Compact binary edges). For a binary metagraph, replace each edge’s anchor and roles by the statement $h(x) = (h(y), \text{lab}(x), h(z))$. Use labels outside the reserved predicate set; then an edge row has two anchor endpoints, a vertex row retains its vertex shape, and membership and annotation shapes remain distinct. The compact decoder reads each such row as one binary edge. This encoding retains the component clocks and direct endpoint dependencies. Snapshot commutation and structural closure apply to it, but LR realisability additionally requires an acyclic edge-dependency graph. Cyclic binary metagraphs are representable as static compact LBGs, but cannot be inserted from an empty store under LR.

Using $(n_y, \text{lab}(x), n_z)$ instead loses vertex-occurrence dependencies and is outside the vertex-lifetime lifting claim. In the general encoding, retracting an endpoint may leave its incident edge

anchor after role retraction. A metagraph-level deletion must also remove incident edge anchors and their dependents; raw statement cascade alone does not preserve general normal form.

6.3 The lifting theorem

Theorem 10 (Bitemporal Lifting). *Let H be a well-formed temporal metagraph in the encoding domain of Definition 11. For every transaction time t and valid instant v , the general encoding satisfies*

1. $\text{dec}(V_t^v(\text{enc}(H))) \cong H(t, v)$;
2. $V_t^v(\text{enc}(H))$ is grounded and is in temporal normal form;
3. if every component lifetime is non-empty, the encoded history is realisable by transactions obeying LR.

For the compact binary encoding, (1) and (2) hold with its compact normal form; (3) additionally requires acyclic edge dependencies and non-empty lifetimes. For an ill-formed binary history encoded compactly, decoding the ground core of its snapshot gives the largest dependency-closed component subset. This last claim is not made for the general anchor-and-role encoding.

Proof. For the general encoding, anchors and component rows inherit their clocks; all roles of an edge share its anchor's clocks. Hence a snapshot retains the entire encoding of each live component. Well-formedness retains the required anchors, so the snapshot equals $\text{enc}(H(t, v))$ and Theorem 9 applies. This also proves closure and normal form. For (3), at each transaction insert all new anchors first, then their roles, memberships and annotations; anchors reference no statements. Before retracting an anchor, retract component rows and roles ending at that transaction, then the anchors ending there. Lifetime containment guarantees each referent is live at insertion and no longer-lived referencing row remains at retraction. The non-empty-lifetime assumption ensures that a dependency is alive at its dependent's addition time. Empty lifetimes have no snapshot effect, but can violate LR scheduling when their referents have not yet been inserted; they are therefore excluded from the realisability claim. This is a model-level schedule, not a claim that every reserved row is writable through the current public API.

In compact form, each component's references are exactly its dependencies. The same snapshot argument applies. If dependencies are acyclic, insert new edge occurrences in referent-first topological order and retract in reverse order. Lifetime containment alone does not supply such an order for a cycle. For an ill-formed compact snapshot, the core retains precisely the components whose transitive dependencies remain, proving maximality by Lemma 1. General edge anchors do not reference their endpoints, so this last argument fails for the general encoding. $\square$

This theorem transfers snapshots under explicit well-formedness conditions. It does not establish that arbitrary statement corrections or cascades preserve metagraph normal form. General n-ary edges require component-level integrity validation in addition to reference closure.

6.4 Consequences for metagraph operations

Corollary 11 (Unfold and nesting in time). *For a well-formed H , the members of a container c in $H(t, v)$ are the subjects of the **sys:inGraph** statements with object $h(c)$ in $V_t^v(\text{enc}(H))$, and the transitive closure of membership at (t, v) is the answer of a regular path query over $\text{enc}(H)$ evaluated in that view.*

Corollary 12 (Containers retract, contents stay). *Retracting the anchor $h(c)$ of a container retracts every membership into c , every annotation on c and every statement resting on those, and retracts a member anchor only if that anchor transitively references $h(c)$. This is a statement-level cascade claim; preserving general metagraph normal form can require further edge removals.*

Proof. By Theorem 4, the cascade is $\uparrow_X h(c)$. A membership into c has $h(c)$ as its object and so lies in it. A member's anchor lies in it only if it reaches $h(c)$ through references. The membership row references both anchors; it creates no outgoing reference from the member anchor to the container. $\square$

Corollary 13 (Fold is a non-destructive write). *Folding a set F of atoms into a container c at transaction t inserts one membership per atom of F , with $c \notin F$, live anchors and valid intervals contained in those of both endpoints. It changes no view $V_{t'}$ with $t' < t$, and for every $t'' \geq t$ before any later change and every v in the intersection of the intervals involved, unfolding c at (t'', v) returns F together with any earlier members.*

Proof. By Theorem 3 and Corollary 11. $\square$

Proposition 14 (Size). $|\text{enc}(M)| = |V| + |E| + \sum_{x \in E} (|\text{src}(x)| + |\text{tgt}(x)|) + |\mu| + |\alpha|$, linear in the size of M ; with compact binary edges, a binary edge costs one statement.

For an admissible correction of a compact container edge, internal memberships retained in R follow its new identifier, while member anchors retain their identifiers unless they are themselves in the cascade. A correction of a general edge anchor must also keep its roles coherent with the patched label and clocks; Theorem 8 alone does not validate that metagraph-specific operation.

7 Paths Through Time

Layered graphs make paths across layers natural: a walk can go from a belief to the fact it rests on, to the source of that fact, through virtual hops from a statement to its subject or object. On top of the two clocks, a path can also be required to respect valid time. A walk is *time-respecting* from start time τ_0 if it can traverse each statement at some instant inside its valid interval, never going back [26, 31]. With τ the walk's current time, a hop over interval $[a, b)$ is allowed iff $b > \tau$, and moves τ to $\max(\tau, a)$.

Proposition 15 (Earliest arrival with a hop bound). *Fix a finite product of a graph view at transaction time t and a regular-path automaton, a start instant τ_0 , and a finite hop bound K . Waiting is allowed and traversal takes zero time. Every hop, including a virtual hop, counts toward K ; virtual hops preserve the current instant. Let $B_k(z)$ be the least arrival at product state z by walks of at most k hops, with $+\infty$ for unreachable states. Initialise the start to τ_0 and the other states to $+\infty$. Then*

$$B_{k+1}(z) = \min \left(B_k(z), \min_{w \xrightarrow{[a,b)} z, B_k(w) < b} \max(B_k(w), a) \right)$$

(with the analogous time-preserving update for virtual hops) returns the earliest arrival at every accepting end within K hops. It may be implemented by hop layers, expanding only strict improvements relative to completed earlier layers; updates in a new layer must not overwrite the saved labels of the layer being expanded.

Proof. Induct on k . A walk of at most $k+1$ hops either already has at most k , or has a last hop appended to a walk of at most k . The hop rule is monotone: a feasible hop at a later instant is feasible at an earlier one and ends no later. Therefore the best prefix label suffices in the recurrence. For pruning, a label no better than one from a completed earlier layer is dominated in both time and hop usage; that earlier label has already propagated its feasible extensions. A later layer's improvement cannot retroactively dominate a prefix with more remaining hop budget, so saved current-frontier labels are expanded. Taking the minimum over accepting automaton states proves the end-arrival claim. $\square$

Keeping only earliest arrival in an arbitrary priority-by-time search is insufficient under a hop bound: a later arrival using fewer hops may reach an end that an earlier arrival cannot. The repository’s timed reachability search uses saved hop frontiers, matching the required scheduling discipline. Existing property tests compare it with brute-force bounded walks. The hop rule follows standard temporal path semantics [49]; the explicit bound and product-state scheduling are part of this proposition. Snapshot lifting alone does not prove journey equivalence for an arbitrary general n -ary edge traversal.

8 Realisation and Conformance

8.1 Tiramemsu

The model is implemented in Tiramemsu, an embedded graph database written in Rust over SQLite, queried with SPARQL and Cypher over one store. A statement is one row ($eid, s, p, o, t_{add}, t_{ret}, v_{from}, v_{to}, ret_kind$). Every identifier is a 64-bit integer with a 4-bit type tag in the low bits, so statements, nodes, transactions and small literals share one column type and stay compact as SQLite varints. Permutation indexes are split into partial indexes over live rows, which serve the current view, and history indexes ordered by t_{add} descending, which support as-of scans with addition-time bounds and residual lifetime filters. Triggers reject deletion, replacement, and immutable-content updates, and allow a single retraction. Format 3 adds date guards: statement addition and retraction use the latest transaction record above the committed watermark. Raw SQL writers must follow the transaction protocol; deliberate schema or metadata tampering is outside these guards. Named graphs, including graphs named by statements, are membership rows in the same table; a graph costs one row and its index entries per member, and selecting a graph is a seek on the predicate-object index.

Several results of this paper are checked by property-based tests over random operation sequences and random layered graphs with reference cycles and memberships: replay equivalence (Theorem 3); equality of the read-only dependents, the dry-run cascade and the reverse path query (Theorem 4); round-tripping of fact bundles; and time-respecting search against enumeration (Section 7).

8.2 Conformance repairs

Earlier revisions retained four counterexamples against the engine. This revision implements repairs and changes the retained probes to assert that those cases are prevented. The probes use a fresh temporary database, and integration regressions run on both the full SQLite host and the minimal host.

1. **Retracted and missing endpoints.** Ordinary assertions and creates now require statement-valued subjects and objects to name live rows. Checks run before writing and again after cardinality replacement, which can retract a proposed endpoint through its cascade. Unknown and retracted targets are rejected. The engine’s reserved `sys:supersedes` lineage may name allocated historical occurrences; it cannot name missing ones. There is no general user-defined lineage registry in this implementation.
2. **Dropped membership layers.** Before allocating copies, correction selects foreign memberships and propagates exclusion through their structural dependents inside the cascade. Only retained rows receive fresh identifiers. A dropped root is rejected. Layers of dropped memberships therefore remain historical rather than pointing to copies that were never inserted.
3. **Patched and external references.** A root object naming a retained cascade member is substituted to its fresh copy. Direct self-reference and patched references to dropped rows

are rejected. External structural targets are checked after schema effects and before replay. This can preserve a complete cyclic retained graph, without promising acyclic correction.

4. **Backdated SQL writes.** Format 3 adds insertion and retraction date triggers. A date must equal the latest transaction record and exceed the committed `meta.last_t` watermark; retraction cannot precede addition. Insert-then-retract within one transaction remains valid. Opening format 1 or 2 files migrates forward atomically without rewriting historical rows.

The original four probes now report prevention. Additional regressions check recursive annotation exclusion, patched-reference substitution, invalid-patch rollback, cardinality side effects and migration. Existing cycles and invalid references in old files are not rewritten or repaired during migration: they still require integrity validation or sanitation before applying complete-carrier results. Raw SQL bypasses other engine checks; the date guards do not establish full referential integrity for arbitrary SQL writes. They assume engine metadata and triggers have not been deliberately modified. The result is evidence for these repaired contracts, not a universal implementation-conformance proof.

8.3 Illustrative agent-memory correction workflow

Consider an assistant storing an employment claim r together with a confidence layer c , a document-evidence layer d , and a membership m in a session container outside the correction cascade. These are distinct statement occurrences; their transaction clocks record when the assistant committed them, while the employment claim’s valid interval records the claimed employment period. If m has an annotation a , dropping m also requires dropping a under structural retention. A later document changes the employer named by r . This example is an application policy, not a measured agent-memory experiment.

The application first separates annotations by meaning. Confidence c and evidence d describe the old employer claim, so copying them unchanged would misrepresent the replacement. Select them for exclusion along with foreign membership m , and compute the induced structural core of the remaining cascade. Require that the root survives. The correction theorem applies to this structurally closed retained set: substitute retained identifiers, patch the root object, retract the original cascade, and add the predecessor lineage. Create fresh confidence and evidence rows only after checking the new claim against the new document. These subsequent assertions obey LR and receive their own transaction and valid intervals. The original evidence remains available in historical views; it is not presented as support for the new employer.

This extends the theorem’s foreign-membership selection policy to an explicitly chosen exclusion set Q : use $R = \gamma_{\Lambda, C}(C \setminus Q)$ and require $r \in R$. The proof uses only structural closure of R , so it applies unchanged. Policy classification and evidence revalidation are application responsibilities. A grounded replacement can still be factually wrong; this workflow establishes reference integrity, not factual accuracy or improved retrieval quality.

8.4 Retained verification results

The revision artifact uses seed 20261003. Its finite checker examines 2,470 reference graphs with at most four occurrences and two referents per occurrence, 38,946 row subsets, and 618,820 intersection cases. It checks 19,053 minimal cascades, 1,730 admissible corrections, 200 general metagraph roundtrips, 8,400 bitemporal snapshots, and 1,000 bounded journeys against enumeration. All checks pass. Four actual-engine probes now verify prevention of the previously reported conformance gaps. These results test selected finite contracts; they do not measure throughput, latency, storage overhead, or agent-memory quality. The retained workflow check also exercises exclusion and revalidation of object-sensitive annotations. Commands and limits are in `paper/artifact/README.md`; hashes and results are in `paper/review-v4/`. The same

assistant performed the revisions and post-revision assessment; an external proof review remains desirable.

9 Discussion

Occurrences, not facts. Identity is per occurrence: two statements may have equal content, and an idempotent assert decides when a new occurrence is created. This is what lets a fact have two episodes with disjoint valid intervals, each with its own annotations. RDF 1.2’s many-to-many reifiers are more flexible; an LBG can model them by a reifier node with ordinary statements, losing the cascade from fact to annotation, which is the point of identity here.

Why valid time is not clamped. One could force a layer’s valid interval inside its base’s on write. We do not, because annotations are often claims *about* a fact made at another time. Theorem 7 gives readers the clamped semantics on demand. A schema flag that requests clamping for a given predicate (for example roles of an n -ary edge, whose validity is the edge’s) would make the reader-side core unnecessary for those predicates.

Branching and merging. The model has one linear transaction axis. Speculative transactions are evaluated and discarded, as in Datomic, and never enter the history. Merging the histories of several agents would need identifiers that are stable across stores; the implementation reserves bits for an origin, but merge semantics, and how γ behaves under merge, are open.

Complexity. Given an in-memory adjacency representation, seed rows of X with a reference outside X and propagate removal along inverse references within X . Each row and reference is visited once, so the core takes $O(|X| + |D_X|)$ time, where D_X includes outgoing boundary references and induced edges. A cascade has the same reachability bound on the visited subgraph. One effective-validity query costs a support traversal and interval intersections; computing it separately for all rows can repeat shared support. On an acyclic history, referent-first dynamic programming computes all effective intervals in linear adjacency time; cycles require strongly connected components first. These are graph-algorithm bounds, not measured SQL latency or storage bounds. The present artifact contains correctness checks, not evidence of performance at 10^4 – 10^7 rows. Incremental materialisation is future work.

Limitations. The theorems are about the data model and its operations, not about query languages. We have not formalised how SPARQL’s default graph or Cypher’s dual view of statements interact with γ . The conformance findings have executable repairs, but not every theorem hypothesis has been audited across every operation. The empirical evidence remains the project’s test suite, not an independent benchmark.

10 Conclusion

Recursive references between statement occurrences make temporal integrity an operational concern. On a complete history, the standard down-set interior characterises closed views and minimal cascades. Lifetime containment and effective validity describe when snapshots retain their dependencies; admissible correction preserves structural closure while recording the predecessor. An explicitly typed metagraph encoding transfers well-formed snapshots, with additional conditions for compact insertion and general edge operations. The four identified implementation gaps have executable repairs. Retained checks exercise their boundary conditions without certifying every operation. These results support an integrity calculus for agent-memory storage, without asserting semantic truth preservation or unmeasured performance.

Availability. Tiramemsu is available at <https://github.com/Volland/tiramemsu> under MIT and Apache-2.0 licences. This preprint is CC BY 4.0. Revision 4 includes the retained artifact in `paper/artifact/`, with commands, fixed model-check seed, engine probes, source hashes and tool versions. The inspected implementation base is commit `fa7880b192a4db3256f0f8cfe2394825a0741e4c`; the workspace also contains uncommitted work, so the artifact manifest hashes the relevant source files. A permanent archived release DOI is still needed before submission.

Evidence limits. The artifact checks finite cases and selected engine scenarios; it is not a machine-checked proof, an independent performance benchmark or an agent-memory quality evaluation. The four reported engine gaps have been repaired and regression-tested; this does not establish universal conformance or sanitize previously stored invalid histories.

References

- [1] Carlos E. Alchourrón, Peter Gärdenfors, and David Makinson. On the logic of theory change: Partial meet contraction and revision functions. *Journal of Symbolic Logic*, 50(2): 510–530, 1985. [10.2307/2274239](#).
- [2] Renzo Angles, Aidan Hogan, Ora Lassila, Carlos Rojas, Daniel Schwabe, Pedro Szekely, and Domagoj Vrgoč. Multilayer graphs: A unified data model for graph databases. In *Proceedings of the 5th ACM SIGMOD Joint International Workshop on Graph Data Management Experiences & Systems (GRADES) and Network Data Analytics (NDA)*, pages 1–6, 2022. [10.1145/3534540.3534696](#).
- [3] Jonathan A. Barmak. *Algebraic Topology of Finite Topological Spaces and Applications*, volume 2032 of *Lecture Notes in Mathematics*. Springer, 2011. [10.1007/978-3-642-22003-6](#).
- [4] Amit Basu and Robert W. Blanning. *Metagraphs and Their Applications*, volume 15 of *Integrated Series in Information Systems*. Springer, New York, 2007. ISBN 978-0-387-37233-4. [10.1007/978-0-387-37234-1](#).
- [5] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. Why and where: A characterization of data provenance. In *Database Theory – ICDT 2001*, volume 1973 of *Lecture Notes in Computer Science*, pages 316–330. Springer, 2001. [10.1007/3-540-44503-X_20](#).
- [6] Jaewook Byun, Sungpil Woo, and Daeyoung Kim. ChronoGraph: Enabling temporal graph traversals for efficient information diffusion analysis over time. *IEEE Transactions on Knowledge and Data Engineering*, 32(3):424–437, 2020. [10.1109/TKDE.2019.2891565](#).
- [7] Jeremy J. Carroll, Christian Bizer, Pat Hayes, and Patrick Stickler. Named graphs, provenance and trust. In *Proceedings of the 14th International Conference on World Wide Web (WWW)*, pages 613–622, 2005. [10.1145/1060745.1060835](#).
- [8] Arnaud Casteigts, Paola Flocchini, Walter Quattrociocchi, and Nicola Santoro. Time-varying graphs and dynamic networks. *International Journal of Parallel, Emergent and Distributed Systems*, 27(5):387–408, 2012. [10.1080/17445760.2012.668546](#). Conference version: ADHOC-NOW 2011, LNCS, pp. 346–359, doi:10.1007/978-3-642-22450-8_27.
- [9] Valeriy Chernenkiy, Yuriy Gapanyuk, Anatoly Nardid, Maria Skvortsova, Anton Gushcha, Yuriy Fedorenko, and Richard Picking. Using the metagraph approach for addressing RDF knowledge representation limitations. In *2017 Internet Technologies and Applications (ITA)*, pages 47–52. IEEE, 2017. [10.1109/ITECHA.2017.8101909](#).

- [10] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory, 2025. URL <https://arxiv.org/abs/2504.19413>.
- [11] Datomic. Datomic documentation: Data model. <https://docs.datomic.com/data-model.html>, 2026. Accessed 2026-10-03.
- [12] Johan de Kleer. An assumption-based TMS. *Artificial Intelligence*, 28(2):127–162, 1986. [10.1016/0004-3702\(86\)90080-9](https://doi.org/10.1016/0004-3702(86)90080-9).
- [13] Ariel Debrouvier, Eliseo Parodi, Matías Perazzo, Valeria Soliani, and Alejandro Vaisman. A model and query language for temporal graph databases. *The VLDB Journal*, 30(5): 825–858, 2021. [10.1007/s00778-021-00675-4](https://doi.org/10.1007/s00778-021-00675-4).
- [14] Jon Doyle. A truth maintenance system. *Artificial Intelligence*, 12(3):231–272, 1979. [10.1016/0004-3702\(79\)90008-0](https://doi.org/10.1016/0004-3702(79)90008-0).
- [15] Johannes Frey, Kay Müller, Sebastian Hellmann, Erhard Rahm, and Maria-Esther Vidal. Evaluation of metadata representations in RDF stores. *Semantic Web*, 10(2):205–229, 2019. [10.3233/SW-180307](https://doi.org/10.3233/SW-180307).
- [16] Yuriy Gapanyuk. The development of the metagraph data and knowledge model. In Valery B. Tarassov, Vadim V. Borisov, and Boris A. Kobrinskii, editors, *Russian Advances in Fuzzy Systems and Soft Computing: Selected Contributions to the 10th International Conference on Integrated Models and Soft Computing in Artificial Intelligence (IMSC-2021)*, volume 2965 of *CEUR Workshop Proceedings*, pages 1–7, 2021. URL <https://ceur-ws.org/Vol-2965/paper01.pdf>.
- [17] Ewout Gelling, George Fletcher, and Michael Schmidt. Statement graphs: Unifying the graph data model landscape. In *Database Systems for Advanced Applications (DASFAA 2024)*, Lecture Notes in Computer Science, pages 364–376. Springer, 2024. [10.1007/978-981-97-5575-2_27](https://doi.org/10.1007/978-981-97-5575-2_27).
- [18] Ben Goertzel. Folding and unfolding on metagraphs, 2020. URL <https://arxiv.org/abs/2012.01759>.
- [19] Todd J. Green, Grigoris Karvounarakis, and Val Tannen. Provenance semirings. In *Proceedings of the 26th ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems (PODS)*, pages 31–40, 2007. [10.1145/1265530.1265535](https://doi.org/10.1145/1265530.1265535).
- [20] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, pages 59532–59569, 2024. [10.52202/079017-1902](https://doi.org/10.52202/079017-1902).
- [21] Claudio Gutierrez, Carlos A. Hurtado, and Alejandro Vaisman. Introducing time into RDF. *IEEE Transactions on Knowledge and Data Engineering*, 19(2):207–218, 2007. [10.1109/TKDE.2007.34](https://doi.org/10.1109/TKDE.2007.34).
- [22] Olaf Hartig. Foundations of RDF* and SPARQL* (an alternative approach to statement-level metadata in RDF). In Juan Reutter and Divesh Srivastava, editors, *Proceedings of the 11th Alberto Mendelzon International Workshop on Foundations of Data Management (AMW)*, volume 1912 of *CEUR Workshop Proceedings*, 2017. URL <https://ceur-ws.org/Vol-1912/paper12.pdf>.

- [23] Olaf Hartig, Pierre-Antoine Champin, Andy Seaborne, and Gregg Kellogg. RDF 1.2 concepts and abstract data model. W3C Candidate Recommendation Snapshot, April 2026. URL <https://www.w3.org/TR/2026/CR-rdf12-concepts-20260407/>. 7 April 2026; cited names are editors; Gregg Kellogg served until 6 September 2025.
- [24] Daniel Hernández, Aidan Hogan, and Markus Krötzsch. Reifying RDF: What works well with Wikidata? In *Proceedings of the 11th International Workshop on Scalable Semantic Web Knowledge Base Systems (SSWS)*, volume 1457 of *CEUR Workshop Proceedings*, pages 32–47, 2015. URL https://ceur-ws.org/Vol-1457/SSWS2015_paper3.pdf.
- [25] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d’Amato, Gerard de Melo, Claudio Gutierrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, Axel-Cyrille Ngonga Ngomo, Axel Polleres, Sabbir M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan Sequeda, Steffen Staab, and Antoine Zimmermann. Knowledge graphs. *ACM Computing Surveys*, 54(4):71:1–71:37, 2021. [10.1145/3447772](https://doi.org/10.1145/3447772).
- [26] Petter Holme and Jari Saramäki. Temporal networks. *Physics Reports*, 519(3):97–125, 2012. [10.1016/j.physrep.2012.03.001](https://doi.org/10.1016/j.physrep.2012.03.001).
- [27] Jiamin Hou, Zhanhao Zhao, Zhouyu Wang, Wei Lu, Guodong Jin, Dong Wen, and Xiaoyong Du. AeonG: An efficient built-in temporal support in graph databases. *Proceedings of the VLDB Endowment*, 17(6):1515–1527, 2024. [10.14778/3648160.3648187](https://doi.org/10.14778/3648160.3648187).
- [28] Borislav Iordanov. HyperGraphDB: A generalized graph database. In *Web-Age Information Management (WAIM 2010 Workshops)*, volume 6185 of *Lecture Notes in Computer Science*, pages 25–36. Springer, 2010. [10.1007/978-3-642-16720-1_3](https://doi.org/10.1007/978-3-642-16720-1_3).
- [29] Christian S. Jensen, Michael D. Soo, and Richard T. Snodgrass. Unifying temporal data models via a conceptual model. *Information Systems*, 19(7):513–547, 1994. [10.1016/0306-4379\(94\)90013-2](https://doi.org/10.1016/0306-4379(94)90013-2).
- [30] Christian S. Jensen, Curtis E. Dyreson, Michael Böhlen, James Clifford, Ramez Elmasri, Shashi K. Gadia, Fabio Grandi, Pat Hayes, Sushil Jajodia, Wolfgang Käfer, Nick Kline, Nikos Lorentzos, Yannis Mitsopoulos, Angelo Montanari, Daniel Nonen, Elisa Peressi, Barbara Pernici, John F. Roddick, Nandlal L. Sarda, Maria Rita Scalas, Arie Segev, Richard T. Snodgrass, Michael D. Soo, Abdullah Tansel, Paolo Tiberio, and Gio Wiederhold. The consensus glossary of temporal database concepts – February 1998 version. In Opher Etzion, Sushil Jajodia, and Suryanarayana Sripada, editors, *Temporal Databases: Research and Practice*, volume 1399 of *Lecture Notes in Computer Science*, pages 367–405. Springer, 1998. [10.1007/BFb0053710](https://doi.org/10.1007/BFb0053710).
- [31] David Kempe, Jon Kleinberg, and Amit Kumar. Connectivity and inference problems for temporal networks. *Journal of Computer and System Sciences*, 64(4):820–842, 2002. [10.1006/jcss.2002.1829](https://doi.org/10.1006/jcss.2002.1829).
- [32] Mikko Kivelä, Alex Arenas, Marc Barthélemy, James P. Gleeson, Yamir Moreno, and Mason A. Porter. Multilayer networks. *Journal of Complex Networks*, 2(3):203–271, 2014. [10.1093/comnet/cnu016](https://doi.org/10.1093/comnet/cnu016).
- [33] Krishna Kulkarni and Jan-Eike Michels. Temporal features in SQL:2011. *ACM SIGMOD Record*, 41(3):34–43, 2012. [10.1145/2380776.2380786](https://doi.org/10.1145/2380776.2380786).
- [34] Ora Lassila, Michael Schmidt, Olaf Hartig, Brad Bebee, Dave Bechberger, Willem Broekema, Ankesh Khandelwal, Kelvin Lawrence, Carlos Manuel Lopez Enriquez, Ronak Sharda, and Bryan Thompson. The OneGraph vision: Challenges of breaking the graph model lock-in. *Semantic Web*, 14(1):125–134, 2023. [10.3233/SW-223273](https://doi.org/10.3233/SW-223273).

- [35] Maria Massri, Zoltan Miklos, Philippe Raipin, and Pierre Meye. Clock-G: A temporal graph management system with space-efficient storage technique. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*, pages 2263–2276, 2022. [10.1109/ICDE53745.2022.00215](https://doi.org/10.1109/ICDE53745.2022.00215).
- [36] Vera Zaychik Moffitt and Julia Stoyanovich. Temporal graph algebra. In *Proceedings of the 16th International Symposium on Database Programming Languages (DBPL)*, pages 10:1–10:12, 2017. [10.1145/3122831.3122838](https://doi.org/10.1145/3122831.3122838).
- [37] Vinh Nguyen, Olivier Bodenreider, and Amit Sheth. Don’t like RDF reification? Making statements about statements using singleton property. In *Proceedings of the 23rd International Conference on World Wide Web (WWW)*, pages 759–770, 2014. [10.1145/2566486.2567973](https://doi.org/10.1145/2566486.2567973).
- [38] Fabrizio Orlandi, Damien Graux, and Declan O’Sullivan. Benchmarking RDF metadata representations: Reification, singleton property and RDF*. In *2021 IEEE 15th International Conference on Semantic Computing (ICSC)*, pages 233–240, 2021. [10.1109/ICSC50631.2021.00049](https://doi.org/10.1109/ICSC50631.2021.00049).
- [39] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems, 2023. URL <https://arxiv.org/abs/2310.08560>.
- [40] Volodymyr Pavlyshyn. *Metagraph for AI Agents: Metagraphs for Agentic AI: Beyond Triples, Beyond Hypergraphs*. Leanpub, 2026. URL <https://leanpub.com/metagraphforaiagents>. Self-published e-book; cited 2026 edition last updated 24 March 2026; original release year not established.
- [41] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory, 2025. URL <https://arxiv.org/abs/2501.13956>.
- [42] Christopher Rost, Kevin Gomez, Matthias Täschner, Philip Fritzsche, Lucas Schons, Lukas Christ, Timo Adameit, Martin Junghanns, and Erhard Rahm. Distributed temporal graph analytics with GRADOOP. *The VLDB Journal*, 31(2):375–401, 2022. [10.1007/s00778-021-00667-4](https://doi.org/10.1007/s00778-021-00667-4).
- [43] Richard T. Snodgrass. *Developing Time-Oriented Database Applications in SQL*. Morgan Kaufmann, San Francisco, 1999. ISBN 1-55860-436-7.
- [44] Richard T. Snodgrass and Ilsoo Ahn. A taxonomy of time in databases. In *Proceedings of the 1985 ACM SIGMOD International Conference on Management of Data*, pages 236–246. ACM, 1985. [10.1145/318898.318921](https://doi.org/10.1145/318898.318921). Also in ACM SIGMOD Record 14(4), doi:10.1145/971699.318921.
- [45] Alfred Tarski. A lattice-theoretical fixpoint theorem and its applications. *Pacific Journal of Mathematics*, 5(2):285–309, 1955. [10.2140/pjm.1955.5.285](https://doi.org/10.2140/pjm.1955.5.285).
- [46] Denny Vrandečić and Markus Krötzsch. Wikidata: A free collaborative knowledgebase. *Communications of the ACM*, 57(10):78–85, 2014. [10.1145/2629489](https://doi.org/10.1145/2629489).
- [47] Domagoj Vrgoč, Carlos Rojas, Renzo Angles, Marcelo Arenas, Diego Arroyuelo, Carlos Buil-Aranda, Aidan Hogan, Gonzalo Navarro, Cristian Riveros, and Juan Romero. MillenniumDB: An open-source graph database system. *Data Intelligence*, 5(3):560–610, 2023. [10.1162/dint_a_00229](https://doi.org/10.1162/dint_a_00229). Preprint: arXiv:2111.01540.

- [48] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Long-MemEval: Benchmarking chat assistants on long-term interactive memory, 2024. URL <https://arxiv.org/abs/2410.10813>.
- [49] Huanhuan Wu, James Cheng, Silu Huang, Yiping Ke, Yi Lu, and Yanyan Xu. Path problems in temporal graphs. *Proceedings of the VLDB Endowment*, 7(9):721–732, 2014. [10.14778/2732939.2732945](https://doi.org/10.14778/2732939.2732945).
- [50] XTDB. XTDB documentation: Time in XTDB. <https://docs.xtdb.com/about/time-in-xtdb.html>, 2026. Accessed 2026-10-03.
- [51] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic memory for LLM agents, 2025. URL <https://arxiv.org/abs/2502.12110>.